

Самостоятельная работа №1

«Обобщенная архитектура баз данных»

Кротов Иван Сергеевич, группа ИВТ-2

Инвариантная часть

Задание 1.1 (Таблица «Типы данных и объекты СУБД MySQL»)

№	Тип данных в MySQL	Описание / характеристики
1	INT / INTEGER	Тип данных, используемый для хранения целых чисел.
2	FLOAT	Тип данных для чисел с плавающей точкой. Возможны небольшие погрешности округления.
3	DECIMAL	Тип данных для чисел с фиксированной точкой. Медленнее FLOAT, но полностью исключает погрешности округления.
4	VARCHAR	Строковый тип данных переменной длины с заданным лимитом символов.
5	CHAR	Строковый тип данных строго фиксированной длины. Если строка меньше заданной длины, она автоматически дополняется пробелами.
6	TEXT	Тип данных для хранения больших объемов текстовой информации. Используется, когда лимита VARCHAR недостаточно.
7	DATE	Тип данных для хранения даты в формате ГГГГ-ММ-ДД.
8	DATETIME	Тип данных для хранения даты. В отличие от DATE, хранит дату с точностью до секунды.
9	BOOLEAN	Логический тип данных, принимающий только два значения: TRUE и FALSE (Истина и Ложь, 1 и 0).
10	TABLE	Базовый объект реляционной СУБД для хранения данных в строках и столбцах.
11	PRIMARY KEY	Первичный ключ, уникальный идентификатор для записей в таблице.
12	FOREIGN KEY	Внешний ключ для связи таблиц между собой.
13	INDEX	Объект, предназначенный для оптимизации поисковых запросов и ускорения сортировки данных.
14	VIEW	Виртуальная таблица, не хранящая данные физически, формируемая на основе SQL-запроса.

Задания 1.2 и 1.3 (Презентации «Ведущие производители СУБД» и «Этапы развития СУБД»)

Презентации по этим 2 заданиям выложены вместе с этим отчётом по самостоятельной работе соответственно в файлах:

1. Ведущие производители СУБД.pptx
2. Этапы развития СУБД.pptx

Вариативная часть

Задание 1.4 (Развертывание MongoDB с помощью Docker)

Для развёртывания MongoDB с помощью докер воспользуемся командой запуска контейнера MongoDB:

docker run -d --name mongodb -p 27017:27017 mongo

Разберёмся, как работает эта команда:

1. **docker run** – это запуск контейнер
2. **-d** говорит о запуске контейнера в фоновом режиме
3. **--name mongodb** – это имя контейнера
4. **-p 27017:27017** - проброс порта MongoDB
5. **mongo** – используемый образ MongoDB

Далее проверим, что контейнер запущен, с помощью команды **docker ps** (результаты этих двух шагов видны на рисунке 1).

```
C:\Users\Ivan>docker run -d --name mongodb -p 27017:27017 mongo
Unable to find image 'mongo:latest' locally
latest: Pulling from library/mongo
Digest: sha256:d6566e93e6a913cdb622ebe34e0ae7937d50efa60e92363fb4a84404dc890415
Status: Downloaded newer image for mongo:latest
3465822d2c532ea9d996da20e00004ef4f4b026ea00ac6ab0e63505d3b042469

C:\Users\Ivan>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
3465822d2c53   mongo    "docker-entrypoint.s..." 10 seconds ago Up 9 seconds  0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp   mongodb
```

Рисунок 1

Теперь подключимся к MongoDB командой:

docker exec -it mongodb mongosh

Разберёмся, как работает эта команда:

1. **docker exec** – это запуск новой команды внутри работающего контейнера
2. **-it** (или аналогично **-i -t**): **-i** держит стандартный ввод открытым для ввода команд с клавиатуры в контейнер, а **-t** добавляет цветной текст, автодополнение и приглашение к вводу
3. **mongodb** - имя (или ID) конкретного контейнера, в который мы входим
4. **mongosh** – утилита, которая запускается внутри контейнера для управления БД через команды

Результат подключения к MongoDB представлен на рисунке 2.



```
C:\Users\Ivan>docker exec -it mongodb mongosh
Current Mongosh Log ID: 6a130b4818fe4775d09df8a2
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:  8.2.9
Using Mongosh:  2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.
```

Рисунок 2

Теперь обратимся к какой-нибудь базе данных (при её отсутствии, она создастся автоматически), например, к базе `test_db`:

use test_db

Далее добавим какой-нибудь элемент. Например, я хочу добавить запись, пользователя с именем Ivan, которому 20 лет, в коллекцию `users` (которая, опять же, создастся автоматически, если не существовала ранее):

db.users.insertOne({name: "Ivan", age: 20})

Результат этих действий виден на рисунке 3.

```
test> use test_db
switched to db test_db
test_db> db.users.insertOne({name: "Ivan", age : 20})
{
  acknowledged: true,
  insertedId: ObjectId('6a13330b18fe4775d09df8a4')
}
test_db> _
```

Рисунок 3

Попробуем получить данные из коллекции. Воспользуемся командой **db.users.find()** , после чего увидим содержимое коллекции users, то есть одну запись. Результат выполнения команды показан на рисунке 4.

```
test_db> db.users.find()
[
  { _id: ObjectId('6a13330b18fe4775d09df8a4'), name: 'Ivan', age: 20 }
]
test_db> _
```

Рисунок 4